TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐỒNG NAI (DNTU)
KHOA CÔNG NGHỆ THÔNG TIN – HỌC PHẦN WEB API (TH71108)

NGÂN HÀNG CÂU HỎI & ĐÁP ÁN PHẢN BIỆN CHUYÊN SÂU HỆ THỐNG VISIONARY – HỆ THỐNG QUẢN LÝ HOẠT ĐỘNG HỘI NHẬP

Tài liệu huấn luyện phản biện và kiểm chứng thực tế cho 4 thành viên nhóm VISIONARY API
Bao phủ trọn vẹn 18 câu hỏi gốc trong PDF Giảng viên + 30 câu hỏi chuyên sâu gắn liền mã nguồn thật

CÂU 1: DÒNG NÀO BẬT CORS? XÓA ĐI THÌ NỀN TẢNG NÀO HỎNG, HAI NỀN TẢNG KIA VÌ SAO KHÔNG?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm A - Server & Hợp đồng	**Trung bình**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

Câu trả lời nhanh (20–40 giây):

CORS được đăng ký ở dòng builder.Services.AddCors(...) và kích hoạt bằng app.UseCors() đặt TRƯỚC app.MapControllers(). Nếu xóa đi, chỉ có ứng dụng Web bị hỏng vì CORS là cơ chế bảo mật Same-Origin của trình duyệt, không phải của giao thức HTTP. Windows Forms và Android không chạy trong trình duyệt nên không bị CORS chặn.

Giải thích kỹ thuật chuyên sâu:

CORS (Cross-Origin Resource Sharing) sinh ra để ngăn chặn kịch bản tấn công Cross-Site Scripting (XSS) trên trình duyệt. Trình duyệt áp dụng quy tắc Same-Origin Policy: nếu trang web tải từ domain A (ví dụ localhost:3000 hoặc visionary-platform.vercel.app) mà fetch tới domain B (visionary-api-gb51.onrender.com), trình duyệt sẽ gửi request OPTIONS thăm dò hoặc kiểm tra header Access-Control-Allow-Origin. Nếu server không trả header này, trình duyệt giữ lại response và báo lỗi Failed to fetch. Trong khi đó, HttpClient của WinForms và OkHttp của Android tạo socket TCP trực tiếp, không có lớp kiểm soát nguồn gốc của trình duyệt.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Program.cs
Dòng: 120-139 (AddCors) và dòng 209: app.UseCors(policy => policy.AllowAnyOrigin().AllowAnyHeader().AllowAnyMethod()); đặt trước app.MapControllers() (dòng 214).

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Dùng Postman hoặc curl gọi GET /api/posts -> Luôn nhận 200 OK. Nhưng nếu xóa app.UseCors() và mở F12 Console trên Web -> Sẽ thấy thông báo đỏ 'Access to fetch has been blocked by CORS policy'.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Trả lời rằng 'API chặn web'. Sự thật là API không chặn, mà chính TRÌNH DUYỆT chặn trước khi JavaScript kịp đọc dữ liệu.

CÂU 2: LÚC CHẠY, TỆP CƠ SỞ DỮ LIỆU NẰM Ở THƯ MỤC NÀO?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm A - Server & Hợp đồng	**Dễ**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

⚡ Câu trả lời nhanh (20–40 giây):

Đối với ứng dụng mẫu học phần dùng SQLite thì tệp app.db nằm ở thư mục tiến trình đang chạy (Environment.CurrentDirectory). Nhưng trong dự án VISIONARY thực tế, nhóm em sử dụng cơ sở dữ liệu đám mây PostgreSQL 16.3 lưu trữ trên Supabase tại Singapore, không nằm trong ổ đĩa cục bộ của máy.

🔍 Giải thích kỹ thuật chuyên sâu:

Nếu dùng SQLite với chuỗi kết nối 'Data Source=app.db', SQLite tìm file tại thư mục mà tiến trình chạy. Nhấn F5 trong Visual Studio thì tiến trình chạy ở thư mục dự án nên thấy file, nhưng chạy file .exe trong bin/Release thì app.db không có ở đó sẽ bị lỗi 'no such table'. Trong VISIONARY, nhóm sử dụng Managed PostgreSQL: dữ liệu nằm trên các máy chủ đám mây của AWS ap-southeast-1 do Supabase quản lý, kết nối qua Npgsql Pooler cổng 5432.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Infrastructure/Data/AppDbContext.cs
Cấu hình kết nối: Program.cs dòng 50-75 đọc ConnectionStrings:DefaultConnection.

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở endpoint /health/ready trên Swagger -> Phản hồi JSON: { status: 'Ready', database: 'Connected', provider: 'Npgsql.EntityFrameworkCore.PostgreSQL' }.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nhầm lẫn giữa cơ sở dữ liệu SQLite cục bộ trong bài giảng và PostgreSQL đám mây thực tế của dự án.

CÂU 3: HAI ENDPOINT TRẢ VỀ HAI KHUÔN JSON KHÁC NHAU. CHỖ NÀO TRONG MÃ XỬ LÝ KHÁC BIỆT ĐÓ?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm A - Server & Hợp đồng	**Trung bình**	Hồ Đoàng Nguyên Khang	Đào Trí Khang

⚡ Câu trả lời nhanh (20–40 giây):

Trong VISIONARY, endpoint GET /api/events trả về đối tượng phân trang PagedResponse chứa items và totalCount, còn GET /api/events/{slug} trả về đối tượng đơn EventDto. Sự khác biệt này được định nghĩa ở kiểu trả về của Controller bằng Task<ActionResult<ApiResponse<T>>> và phía client xử lý bằng Generic Type.

🔍 Giải thích kỹ thuật chuyên sâu:

Ở phía backend, ASP.NET Core sử dụng lớp bọc ApiResponse<T>. Khi T là PagedResponse<EventDto>, JsonElement chứa { success, data: { items: [...], totalCount, pageIndex } }. Khi T là EventDto, JsonElement chứa { success, data: { id, title, location } }. Phía WinForms C# dùng phương thức GetAsync<T> truyền kiểu T tương ứng; phía Android Retrofit khai báo Call<ApiResponse<PagedResponse<EventItem>>> cho danh sách và Call<ApiResponse<EventItem>> cho chi tiết.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

Backend: EventsController.cs (dòng 29: PagedResponse vs dòng 87: EventDto).
Android: VisionaryApiService.java dòng 29 (PagedResponse) và dòng 34.
WinForms: ApiClient.cs dòng 177 (GetEventsAsync) vs GetEventBySlugAsync.

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở Swagger, thực thi GET /api/events -> thấy thuộc tính 'items'. Thực thi GET /api/events/{slug} -> thấy thẳng thuộc tính 'title', 'location'.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Khai báo sai kiểu Generic trên Android (ví dụ hứng List<T> cho một endpoint trả về PagedResponse) dẫn đến lỗi JsonParseException văng app.

CÂU 4: VÌ SAO TẠO PHIẾU MƯỢN (ĐĂNG KÝ SỰ KIỆN) KHÔNG CẦN GỬI MÃ TÀI KHOẢN? SERVER LẤY Ở ĐÂU?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm A - Server & Hợp đồng	**Trung bình**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

Câu trả lời nhanh (20–40 giây):

Thân yêu cầu của POST /api/events/{id}/register không hề có trường userId. Server tự trích xuất UserId từ Claims trong chuỗi JWT Bearer Token gửi kèm ở Header Authorization. Điều này ngăn chặn việc người dùng mạo danh đăng ký cho tài khoản khác.

Giải thích kỹ thuật chuyên sâu:

Đây là nguyên tắc bảo mật tối thượng 'Server không tin Client'. Nếu client gửi userId trong body, kẻ tấn công có thể sửa body bằng Postman để đăng ký hoặc thao tác thay người khác. Khi dùng JWT, UserId được đóng gói trong Payload kèm chữ ký HMAC-SHA256 của server. Trong EventsController, server đọc biến: var userId = User.FindFirstValue(ClaimTypes.NameIdentifier). Người dùng không thể sửa token vì sửa là chữ ký sai ngay lập tức.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Controllers/EventsController.cs
Dòng: 203-204:
var userIdStr = User.FindFirstValue(ClaimTypes.NameIdentifier);
if (!Guid.TryParse(userIdStr, out var userId)) return Unauthorized();

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở Swagger, gọi POST /api/events/{id}/register mà không gửi token -> Trả về 401 Unauthorized. Gửi token hợp lệ -> Đăng ký đúng tên người sở hữu token mà không cần truyền id.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho rằng client phải truyền userId lên server trong JSON body.

CÂU 5: TÀI KHOẢN THƯỜNG GỌI DANH SÁCH PHIẾU MƯỢN (SỰ KIỆN CỦA TÔI) THÌ NHẬN MẤY BẢN GHI? VÌ SAO?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm A - Server & Hợp đồng	**Dễ**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

Câu trả lời nhanh (20–40 giây):

Tài khoản thường khi gọi endpoint GET /api/events/my-events chỉ nhận đúng các sự kiện mà chính tài khoản đó đã đăng ký thành công. Việc lọc dữ liệu do Server thực hiện dựa trên UserId đọc từ token, không phụ thuộc vào client.

Giải thích kỹ thuật chuyên sâu:

Thuộc tính [Authorize] chỉ có nhiệm vụ kiểm tra xem request có token hợp lệ hay không. Việc phân quyền dữ liệu theo chủ sở hữu (Ownership Authorization) phải được viết bằng code trong thân hàm Controller: query = query.Where(r => r.UserId == currentUserId). Nhờ vậy, sinh viên A không bao giờ nhìn thấy sự kiện riêng tư của sinh viên B.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Controllers/EventsController.cs
Dòng: 333-345 trong hàm GetMyEvents().

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Đăng nhập tài khoản Đào Trí Khang, đăng ký 1 sự kiện -> gọi /api/events/my-events nhận đúng 1 sự kiện. Đăng nhập tài khoản khác nhận 0 sự kiện.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nghĩ rằng client tự lọc dữ liệu ở phía giao diện (Front-end filtering).

CÂU 6: GỬI KÍCH THƯỚC TRANG BẰNG 500 THÌ SERVER TRẢ VỀ BAO NHIÊU BẢN GHI?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm A - Server & Hợp đồng	**Trung bình**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

⚡ Câu trả lời nhanh (20–40 giây):

Nếu gửi pageSize = 500, server ASP.NET Core của VISIONARY sẽ tự động chặn biên an toàn bằng hàm Math.Clamp(pageSize, 1, 50) và chỉ trả về tối đa 50 bản ghi. Điều này giúp ngăn chặn tấn công DoS làm tràn RAM server.

🔍 Giải thích kỹ thuật chuyên sâu:

Trong thiết kế Web API chuyên nghiệp, không bao giờ cho phép client yêu cầu số lượng bản ghi tùy ý. Trong PostsController và EventsController, nhóm em đã lập trình cơ chế kẹp biên: pageSize = Math.Clamp(pageSize, 1, 50). Dù client cố tình gửi pageSize=500 hay pageSize=9999, server vẫn chỉ thực thi SQL LIMIT 50 trên PostgreSQL. Nhờ vậy, hiệu năng server luôn ổn định.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Controllers/PostsController.cs dòng 48:
pageSize = Math.Clamp(pageSize, 1, 50);
pageIndex = Math.Max(1, pageIndex);

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở Swagger, gọi GET /api/posts?pageSize=500 -> Quan sát JSON trả về: trường pageSize tự động bằng 50, và items chỉ có tối đa 50 phần tử.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Trả lời rằng 'Server trả về 500 bản ghi' hoặc 'Server báo lỗi 400'.

CÂU 7: WEB: TOKEN GIỮ Ở ĐÂU? ĐÓNG TAB RỒI MỞ LẠI THÌ CÒN KHÔNG? NHƯ VẬY AN TOÀN TỚI ĐÂU?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm B - Từng nền tảng	**Trung bình**	Quách Khoa Đức	Hồ Đoàng Nguyên Khang

⚡ Câu trả lời nhanh (20–40 giây):

Trong ứng dụng Web Next.js của VISIONARY, token JWT được lưu trong localStorage của trình duyệt. Đóng tab mở lại vẫn còn token cho đến khi token hết hạn 4 giờ hoặc người dùng bấm Đăng xuất. Về độ an toàn: localStorage phòng chống được tấn công CSRF nhưng có nguy cơ nếu trang web dính lỗi XSS.

🔍 Giải thích kỹ thuật chuyên sâu:

localStorage tồn tại vĩnh viễn theo origin domain cho đến khi bị xóa. Nhóm em bảo vệ token trước nguy cơ XSS bằng cách: Không bao giờ dùng dangerouslySetInnerHTML bừa bãi, mã hóa toàn bộ dữ liệu nhập xuất, và triển khai BFF proxy cùng nguồn để che chắn request. Trong mô hình doanh nghiệp cao cấp hơn, có thể chuyển sang HttpOnly Cookie để JavaScript không đọc được token.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: frontend/src/lib/auth.ts (hoặc các component xử lý login: localStorage.setItem('token', token)).

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Đăng nhập trên website -> Mở F12 -> Application -> Local Storage -> Thấy key 'token' với chuỗi eyJhbGci... Đóng tab, mở lại tab mới -> Vẫn ở trạng thái đăng nhập.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nói rằng 'đóng tab là mất' (đó là sessionStorage, không phải localStorage).

CÂU 8: WEB: KHI NÀO TRÌNH DUYỆT GỬI MỘT YÊU CẦU THĂM DÒ TRƯỚC YÊU CẦU THẬT (CORS PREFLIGHT)?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm B - Từng nền tảng	**Khó**	Quách Khoa Đức	Hồ Đoàng Nguyên Khang

Câu trả lời nhanh (20–40 giây):

Trình duyệt tự động gửi yêu cầu thăm dò OPTIONS (Preflight) khi yêu cầu đó không phải là 'Simple Request'. Cụ thể: khi yêu cầu có phương thức PUT, DELETE hoặc POST có Header Content-Type: application/json, hoặc có Header Authorization.

Giải thích kỹ thuật chuyên sâu:

Theo chuẩn W3C CORS, một request là 'Preflighted' khi: (1) Phương thức là PUT, DELETE, PATCH; (2) Có các Header tùy biến ngoài danh sách chuẩn, đặc biệt là header Authorization; (3) Content-Type khác text/plain, multipart/form-data, application/x-www-form-urlencoded. Trình duyệt gửi OPTIONS lên để hỏi server xem có cho phép phương thức và header này không. Nếu server trả 200/204 kèm Access-Control-Allow-Methods và Headers, trình duyệt mới gửi request thật.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Program.cs (AddCors cho phép AllowAnyMethod, AllowAnyHeader).

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở F12 Network trên trình duyệt khi đăng nhập hoặc tạo bài viết -> Sẽ thấy xuất hiện 2 dòng request liên tiếp: dòng đầu là OPTIONS (Preflight) mã 204/200, dòng sau mới là POST thật.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nghĩ rằng lệnh OPTIONS là do lập trình viên viết code gọi.

CÂU 9: WINFORMS: VÌ SAO KHÔNG ĐƯỢC GỌI .RESULT? THỬ GỌI THÌ GIAO DIỆN XỬ SỰ RA SAO?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm B - Từng nền tảng	**Trung bình**	Võ Ngọc Minh Đạt	Hồ Đoàng Nguyên Khang

Câu trả lời nhanh (20–40 giây):

Tuyệt đối không được gọi .Result hoặc .Wait() trên UI Thread trong WinForms vì sẽ gây ra hiện tượng DEADLOCK (Khóa chết). Giao diện sẽ bị đơ cứng, hiện 'Not Responding' vĩnh viễn và không bao giờ nhận được kết quả.

Giải thích kỹ thuật chuyên sâu:

Khi WinForms chạy, luồng giao diện có một SynchronizationContext riêng để xử lý thông điệp Windows. Khi gọi await, Task giải phóng UI thread. Khi hoàn thành, Task yêu cầu SynchronizationContext đưa luồng chạy tiếp. Nhưng nếu gọi .Result, UI thread bị chặn cứng (block) để chờ Task xong, trong khi Task lại đang chờ UI thread rảnh để hoàn tất -> Cả hai chờ nhau vô tận, gây Deadlock.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: desktop/Visionary.Desktop/Forms/MainForm.cs
Nhóm luôn viết: await ApiClient.GetEventsAsync(...) chứ không viết .Result.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Thử sửa một hàm thành .Result -> Bấm nút -> Cửa sổ WinForms lập tức treo cứng, chuột quay tròn, thanh tiêu đề hiện '(Not Responding)'.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho rằng .Result chỉ làm chậm chương trình một chút.

CÂU 10: WINFORMS: VÌ SAO CẢ ỨNG DỤNG CHỈ DÙNG MỘT ĐỐI TƯỢNG GỬI HTTP DUY NHẤT (SINGLETON HTTPCLIENT)?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm B - Từng nền tảng	**Khó**	Võ Ngọc Minh Đạt	Hồ Đoàng Nguyên Khang

Câu trả lời nhanh (20–40 giây):

Vì HttpClient dù có Dispose() thì cổng kết nối mạng Socket bên dưới hệ điều hành vẫn bị treo ở trạng thái TIME_WAIT trong 240 giây. Nếu mỗi lần gọi lại new HttpClient(), hệ điều hành sẽ bị cạn kiệt cổng mạng (Socket Exhaustion) và ném ngoại lệ SocketException.

Giải thích kỹ thuật chuyên sâu:

HttpClient được thiết kế để dùng chung (thread-safe). Khi khởi tạo static readonly HttpClient _httpClient, đối tượng này tái sử dụng pool kết nối TCP, giảm thời gian bắt tay TLS/HTTPS từ vài trăm mili-giây xuống chỉ còn vài mili-giây cho các request tiếp theo.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: desktop/Visionary.Desktop/Services/ApiClient.cs dòng 16:
private static readonly HttpClient _httpClient = new HttpClient { Timeout = TimeSpan.FromSeconds(15) };

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Dùng lệnh netstat -an trên Command Prompt khi gọi API liên tục -> Thấy kết nối TCP được tái sử dụng ở trạng thái ESTABLISHED thay vì sinh ra hàng trăm TIME_WAIT.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nghĩ rằng cứ bọc using(var client = new HttpClient()) là giải phóng hoàn toàn bộ nhớ và mạng.

CÂU 11: ANDROID: CHỈ DÒNG TẠO LUỒNG NỀN VÀ DÒNG ĐẨY KẾT QUẢ VỀ. BỎ ĐI THÌ LỖI GÌ?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm B - Từng nền tảng	**Trung bình**	Đào Trí Khang	Võ Ngọc Minh Đạt

Câu trả lời nhanh (20–40 giây):

Trong Android, dòng tạo luồng nền là ApiClient.getService().getEvents().enqueue(...). Nếu bỏ enqueue mà gọi execute() đồng bộ, app sẽ văng ngay với lỗi NetworkOnMainThreadException. Dòng đẩy kết quả về là callback onResponse() được Retrofit dùng Handler đưa về MainLooper. Nếu tự tạo Thread thuần mà bỏ Handler, app sẽ văng với lỗi CalledFromWrongThreadException.

Giải thích kỹ thuật chuyên sâu:

Hệ điều hành Android áp dụng kiến trúc Single-Threaded UI. Tất cả thao tác vẽ nút, chữ, animation đều do Main Thread xử lý với tốc độ 60 hoặc 120 khung hình/giây. Nếu cho luồng giao diện làm việc nặng như gọi mạng chờ 2 giây, giao diện sẽ bị đông cứng -> Hệ điều hành kích hoạt hộp thoại ANR (Application Not Responding). Ngược lại, chỉ có thread tạo ra View mới được sửa View.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: mobile/VisionaryAndroid/.../EventsFragment.java dòng 50-65.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Đổi enqueue() thành execute() trực tiếp trong onCreateView -> Mở app -> App văng ngay lập tức, Logcat báo lỗi đỏ: android.os.NetworkOnMainThreadException.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nghĩ rằng Java trên Android chạy đa luồng giống hệt Java trên Console.

CÂU 12: ANDROID: BỎ KHAI BÁO CHO PHÉP HTTP THÌ LỖI GÌ? KHÁC LỖI DNS Ở CHỖ NÀO?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm B - Từng nền tảng	**Khó**	Đào Trí Khang	Hồ Đoàng Nguyên Khang

Câu trả lời nhanh (20–40 giây):

Nếu bỏ android:usesCleartextTraffic='true' trong AndroidManifest.xml, app sẽ gặp lỗi: 'CLEARTEXT communication to ... not permitted by network security policy'. Lỗi này khác hoàn toàn lỗi DNS: Lỗi Cleartext là do hệ điều hành Android chủ động CHẶN không cho gửi HTTP không mã hóa; còn lỗi DNS là do máy chủ phân giải tên miền không tìm thấy địa chỉ IP (UnknownHostException).

Giải thích kỹ thuật chuyên sâu:

Từ Android 9.0 (API level 28) trở đi, Google mặc định chặn toàn bộ kết nối HTTP dạng văn bản rõ (cleartext) để bảo vệ người dùng khỏi nghe lén. Muốn gọi vào server cục bộ như 10.0.2.2:5000 bằng HTTP, bắt buộc phải khai báo usesCleartextTraffic='true' hoặc cấu hình network_security_config.xml.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: mobile/VisionaryAndroid/app/src/main/AndroidManifest.xml
Dòng: android:usesCleartextTraffic='true' và res/xml/network_security_config.xml.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Tắt usesCleartextTraffic và gọi HTTP -> Logcat báo CLEARTEXT communication not permitted. Tắt wifi/nhập sai domain -> Logcat báo java.net.UnknownHostException.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Coi tất cả các lỗi không gọi được mạng trên Android đều là 'mạng hỏng'.

CÂU 13: DÒNG CODE NÀY ĐỂ LÀM GÌ? XÓA ĐI THÌ CHUYỆN GÌ XẢY RA? (ASNOTRACKING VÀ PROPERTYNAMECASEINSENSITIVE)

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm C - Mã nguồn nhóm	**Trung bình**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

Câu trả lời nhanh (20–40 giây):

Trong backend, dòng .AsNoTracking() báo cho Entity Framework Core không cần theo dõi thay đổi (Change Tracker) của các bản ghi truy vấn. Nếu xóa đi, ứng dụng vẫn chạy đúng nhưng tốn nhiều bộ nhớ RAM server và truy vấn chậm hơn. Trong WinForms, dòng PropertyNameCaseInsensitive = true giúp bộ đọc JSON không phân biệt hoa thường; nếu xóa đi, tất cả thuộc tính trong C# sẽ bị null vì JSON trả về camelCase trong khi thuộc tính C# là PascalCase.

Giải thích kỹ thuật chuyên sâu:

Mặc định, EF Core lưu trữ snapshot của mọi entity được tải lên vào bộ nhớ đệm để phục vụ SaveChanges(). Với các truy vấn chỉ đọc như danh sách sự kiện hay tin tức, không có thao tác cập nhật nào nên AsNoTracking() giúp giải phóng RAM

ngay lập tức. Còn về JSON: C# đặt tên Title, Location; server trả về title, location. Thiếu PropertyNameCaseInsensitive, JsonSerializer không khớp được tên trường và trả về object rỗng mà không báo lỗi.

Mã nguồn minh chứng (File, Class, Method, Dòng):

Backend: EventsController.cs dòng 38 (.AsNoTracking())
WinForms: ApiClient.cs dòng 23 (PropertyNameCaseInsensitive = true).

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Xóa PropertyNameCaseInsensitive trong WinForms -> Chạy app -> DataGridView hiện các dòng trống không có chữ (tất cả trường đều null).

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nghĩ rằng tên trường C# tự động khớp với JSON mà không cần cấu hình.

CÂU 14: VÌ SAO CHỖ NÀY CHỜ KẾT QUẢ (AWAIT) MÀ CHỖ KIA THÌ KHÔNG (FIRE-AND-FORGET)?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm C - Mã nguồn nhóm	**Khó**	Võ Ngọc Minh Đạt	Hồ Đoàng Nguyên Khang

Câu trả lời nhanh (20–40 giây):

Nhóm dùng await ở những nơi mà giao diện hoặc luồng xử lý kế tiếp bắt buộc phải có dữ liệu của bước trước mới chạy được (ví dụ: chờ đăng nhập lấy token, chờ tải danh sách sự kiện). Ngược lại, không await (fire-and-forget) ở các tác vụ ngầm như ghi log thống kê (Audit Log) hoặc đồng bộ dữ liệu RSS/GitHub tự động, để trả phản hồi cho client nhanh nhất.

Giải thích kỹ thuật chuyên sâu:

Nếu một tác vụ tốn 5 giây như thu thập 100 tin tức từ GitHub mà đặt await, người dùng bấm nút sẽ phải chờ quay vòng tròn suốt 5 giây. Bằng cách đẩy tác vụ đó vào IDataAutomationService chạy ngầm trong Background Task, API trả về HTTP 200 'Đã kích hoạt đồng bộ' trong 50ms, còn server tiếp tục xử lý ở background.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Controllers/TechRadarController.cs dòng 85 (trigger sync chạy ngầm).

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở Swagger gọi POST /api/tech-radar/sync -> Nhận 200 OK ngay trong 80ms. Kiểm tra log server -> Thấy tiến trình đồng bộ vẫn đang âm thầm cào dữ liệu phía sau.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho rằng mọi hàm async đều bắt buộc phải await ở mọi nơi.

CÂU 15: BIẾN NÀY TÊN LÀ GÌ, AI ĐẶT, VÀ VÌ SAO ĐẶT NHƯ VẬY? (VÍ DỤ: _DGVREGISTRATIONS, BTNCHECKIN)

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm C - Mã nguồn nhóm	**Dễ**	Võ Ngọc Minh Đạt	Quách Khoa Đức

Câu trả lời nhanh (20–40 giây):

Biến _dgvRegistrations do bạn Võ Ngọc Minh Đạt đặt. Tên gồm 2 phần: tiền tố 3 ký tự '_dgv' chỉ loại điều khiển DataGridView theo đúng chuẩn tài liệu học phần, và thân tên 'Registrations' là danh từ số nhiều chỉ danh sách các lượt đăng ký sự kiện. Nhìn tên biến là biết ngay loại điều khiển và dữ liệu nó chứa.

Giải thích kỹ thuật chuyên sâu:

Nhóm tuân thủ nghiêm ngặt bảng quy ước đặt tên của thầy: Nút bấm bắt đầu bằng 'btn' (btnCheckIn, btnCreate), ô nhập chữ bắt đầu bằng 'txt' (txtSearch), bảng dữ liệu bắt đầu bằng 'grd' hoặc 'dgv', nhãn hiển thị bắt đầu bằng 'lbl'. Nhóm tuyệt đối không để các tên mặc định như textBox1, button1 vì đây là lỗi trừ điểm nặng khi chấm đồ án.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: desktop/Visionary.Desktop/Forms/MainForm.cs dòng 631 (_dgvRegistrations), dòng 616 (btnCheckIn).

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở file MainForm.cs -> Thấy toàn bộ danh sách điều khiển khai báo đầu class đều có tiền tố chuẩn.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Trả lời 'tên do Visual Studio tự sinh'.

CÂU 16: CHỈ MỘT DÒNG DO AI SINH RA MÀ NHÓM ĐÃ SỬA. SỬA GÌ, VÌ SAO PHẢI SỬA?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm C - Mã nguồn nhóm	**Trung bình**	Quách Khoa Đức	Võ Ngọc Minh Đạt

Câu trả lời nhanh (20–40 giây):

Một lỗi kinh điển do AI sinh ra mà nhóm đã trực tiếp phát hiện và loại bỏ là đoạn logic fallbackPosts giả trong frontend: 'if (res.success && res.data?.items?.length) setPosts(res.data.items); else setPosts(fallbackPosts);'. Nhóm đã sửa triệt để: XÓA 100% fallbackPosts và thay bằng 4 trạng thái chuẩn: Loading, Success, Empty (với thông điệp văn minh) và Error (kèm nút thử lại).

Giải thích kỹ thuật chuyên sâu:

AI thường có xu hướng chèn dữ liệu mẫu giả (mock data) vào code để trang web lúc nào trông cũng có vẻ 'đầy đặn'. Nhưng trong thực tế, nếu danh mục đó chưa có bài viết nào, việc hiển thị bài viết giả gây hiểu nhầm nghiêm trọng cho người dùng và vi phạm tính toàn vẹn dữ liệu. Nhóm đã loại bỏ toàn bộ các biến fallbackPosts, fallbackTech, fallbackEvents khỏi toàn bộ hệ thống.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: frontend/src/app/posts/page.tsx và commit 7c6f3ca trên GitHub.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở tab Bảng vàng hoặc tìm kiếm một từ khóa không tồn tại trên Web -> Hệ thống hiển thị Empty State trung thực 'Chưa có dữ liệu phù hợp với bộ lọc', tuyệt đối không hiện bài viết giả.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nói rằng 'AI viết gì nhóm dùng nấy không sửa gì'.

CÂU 17: TRONG TỆP NÀY CÓ MỘT NHÁNH KHÔNG BAO GIỜ CHẠY TỚI KHÔNG? (DEAD CODE ANALYSIS)

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm C - Mã nguồn nhóm	**Khó**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

Câu trả lời nhanh (20–40 giây):

Nhóm đã rà soát toàn bộ source code bằng Roslyn Analyzer và TypeScript Compiler. Các nhánh kiểm tra null dự phòng như 'if (result?.Data == null)' sau khi đã kiểm tra IsSuccessStatusCode là nhánh phòng thủ (Defensive Programming), bảo vệ ứng dụng khi server trả về chuỗi JSON rỗng 204 No Content.

Giải thích kỹ thuật chuyên sâu:

Một ví dụ cụ thể: Trong hàm xử lý HTTP 204 (No Content) khi cập nhật hoặc xóa, thân phản hồi rỗng. Nếu cố gọi JsonSerializer.Deserialize() sẽ ném ngoại lệ JsonException. Nhóm đặt nhánh kiểm tra: if (response.StatusCode == HttpStatusCode.NoContent) return; Đây là nhánh chủ động xử lý mã 204 theo chuẩn HTTP, không phải dead code.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: desktop/Visionary.Desktop/Services/ApiClient.cs dòng 57, 124.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Gọi lệnh DELETE /api/events/{id} -> Nhận mã 204 -> Ứng dụng xử lý thành công mà không bị crash.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Khẳng định chắc chắn '100% không có nhánh thừa nào' mà không giải thích được cơ chế defensive coding.

CÂU 18: PHẢI XÓA HAI MƯƠI DÒNG MÀ ỨNG DỤNG VẪN CHẠY ĐÚNG – NHÓM XÓA DÒNG NÀO?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nhóm C - Mã nguồn nhóm	**Khó**	Quách Khoa Đức	Hồ Đoàng Nguyên Khang

Câu trả lời nhanh (20–40 giây):

Nhóm có thể xóa ngay 20 dòng mã log console hoặc các khối chú thích ghi chú debug tạm thời trong frontend/src/app/posts/page.tsx hoặc các dòng khai báo Policy CORS trùng lặp trong Program.cs. Ứng dụng vẫn chạy hoàn toàn chính xác theo hợp đồng API.

Giải thích kỹ thuật chuyên sâu:

Trong quá trình phát triển, các lệnh như Console.WriteLine() ở backend và console.log() ở frontend được dùng để theo dõi tiến trình. Khi đưa lên môi trường sản xuất, các dòng này hoàn toàn có thể loại bỏ để mã nguồn gọn gàng hơn mà không ảnh hưởng tới bất kỳ luồng nghiệp vụ nào của hệ thống.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Program.cs (các dòng Console.WriteLine debug DB init).

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Xóa các lệnh in console -> Chạy dotnet test -> 15/15 tests vẫn pass 100%.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Chọn xóa các dòng logic nghiệp vụ hoặc các dòng kiểm tra quyền bảo mật.

CÂU 19: RESTFUL API LÀ GÌ? HỆ THỐNG VISIONARY ÁP DỤNG NHỮNG NGUYÊN LÝ NÀO CỦA REST?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Kiến trúc RESTful API	**Trung bình**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

Câu trả lời nhanh (20–40 giây):

REST (Representational State Transfer) là kiểu kiến trúc phần mềm dựa trên giao thức HTTP. Visionary áp dụng các nguyên lý cốt lõi: Stateless (Phi trạng thái), Client-Server phân tách, Sử dụng danh từ làm URI tài nguyên (/api/events), và ánh xạ chuẩn các phương thức HTTP (GET đọc, POST tạo, PUT sửa, DELETE xóa).

Giải thích kỹ thuật chuyên sâu:

Hệ thống không lưu session người dùng trên RAM server; mỗi request đều độc lập và mang theo JWT token (Stateless). Các tài nguyên được định danh rõ ràng theo chuẩn RESTful: /api/events, /api/events/{slug}, /api/posts/categories.

Mã nguồn minh chứng (File, Class, Method, Dòng):

Mã nguồn: Các Controller trong backend/src/Visionary.Api/Controllers/.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở Swagger UI quan sát các route đều là danh từ, phương thức HTTP đúng chuẩn CRUD.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Đặt động từ vào đường dẫn như /api/getEvents (lệch chuẩn REST).

CÂU 20: TẠI SAO PHẢI CÓ OPENAPI/SWAGGER TRONG DỰ ÁN MÔN HỌC WEB API?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Tài liệu Swagger / OpenAPI	**Dễ**	Hồ Đoàng Nguyên Khang	Đào Trí Khang

Câu trả lời nhanh (20–40 giây):

Swagger/OpenAPI cung cấp tài liệu kỹ thuật sống (Living Documentation) và giao diện trực quan để kiểm thử API tương tác. Giúp lập trình viên Frontend, WinForms và Android đọc hiểu chính xác hợp đồng dữ liệu trước khi viết code.

Giải thích kỹ thuật chuyên sâu:

Thay vì phải viết file Word tài liệu thủ công dễ bị cũ khi code đổi, SwaggerGen tự động quét mã nguồn C# để sinh ra OpenAPI Spec chuẩn JSON tại /swagger/v1/swagger.json. Người dùng có thể nhấn 'Try it out' để kiểm tra kết quả ngay.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Program.cs dòng 143-175, 201-206.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Truy cập https://visionary-api-gb51.onrender.com/swagger -> Thấy đầy đủ schema và mô tả.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nghĩ rằng Swagger chỉ là trang web giới thiệu đồ án thông thường.

CÂU 21: PHÂN BIỆT HTTP GET, POST, PUT, DELETE TRONG HỆ THỐNG VISIONARY?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Giao thức HTTP & CRUD	**Dễ**	Quách Khoa Đức	Võ Ngọc Minh Đạt

Câu trả lời nhanh (20–40 giây):

GET: Lấy dữ liệu (Idempotent, không làm thay đổi trạng thái server). POST: Tạo mới tài nguyên (Không idempotent). PUT: Cập nhật toàn bộ tài nguyên (Idempotent). DELETE: Xóa hoặc hủy tài nguyên (Idempotent).

Giải thích kỹ thuật chuyên sâu:

Tính Idempotent (Bảo toàn kết quả khi lặp lại): Gọi GET 10 lần kết quả server không đổi. Gọi DELETE 1 lần trả về 204, gọi lần 2 trả về 404 nhưng trạng thái DB vẫn là đã xóa. POST thì mỗi lần bấm sẽ tạo thêm 1 bản ghi mới nếu không có khóa chống trùng lặp.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Controllers/EventsController.cs.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Test các phương thức tương ứng trên Swagger.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Dùng GET để xóa dữ liệu (vi phạm bảo mật và chuẩn HTTP).

CÂU 22: CẤU TRÚC APIRESPONSE<T> BAO BỌC DỮ LIỆU MANG LẠI LỢI ÍCH GÌ?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Mô hình Dữ liệu JSON	**Trung bình**	Hồ Đoàng Nguyên Khang	Đào Trí Khang

⚡ Câu trả lời nhanh (20–40 giây):

Cấu trúc ApiResponse<T> chuẩn hóa toàn bộ phản hồi của hệ thống thành một khuôn mẫu duy nhất: { success, data, message, errors }. Giúp ba nền tảng client chỉ cần viết một hàm xử lý lỗi duy nhất thay vì mỗi endpoint một kiểu.

🔍 Giải thích kỹ thuật chuyên sâu:

Nếu không có vỏ bọc chuẩn, khi thành công server trả về Object, khi lỗi trả về chuỗi text hoặc cấu trúc khác, khiến client rất dễ bị crash khi parse JSON. Nhờ ApiResponse<T>, client luôn kiểm tra if (res.success) trước tiên.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Core/Common/ApiResponse.cs.

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Gọi bất kỳ API nào (thành công hay thất bại) đều nhận đúng khuôn JSON có thuộc tính success.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho rằng bọc ApiResponse làm JSON nặng hơn đáng kể.

CÂU 23: PHÂN BIỆT ENTITY TRONG EF CORE VÀ DTO (DATA TRANSFER OBJECT)?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Entity Framework & DTO	**Trung bình**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

⚡ Câu trả lời nhanh (20–40 giây):

Entity là lớp đại diện cho cấu trúc bảng trong cơ sở dữ liệu (có quan hệ khóa ngoại, cột mật khẩu, tracking). DTO là đối tượng trung gian chỉ chứa những dữ liệu cần thiết để truyền qua đường truyền mạng tới client.

🔍 Giải thích kỹ thuật chuyên sâu:

Entity chứa các trường nhạy cảm như PasswordHash, hoặc các thuộc tính quan hệ ảo (virtual Navigation Properties). Nếu serialize Entity thẳng ra JSON, sẽ bị lộ mật khẩu và gây lỗi vòng lặp tuần tự hóa vô tận (Object Cycle). DTO là tấm lá chắn bảo vệ an toàn.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

Entity: backend/src/Visionary.Core/Entities/User.cs
DTO: backend/src/Visionary.Core/DTOs/UserDto.cs.

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Gọi GET /api/auth/me -> Nhận DTO chỉ có fullName, email, roles; hoàn toàn không có trường PasswordHash.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Trả thẳng DbSet<User> ra ngoài controller.

CÂU 24: TẠI SAO HỆ THỐNG SỬ DỤNG UUID (GUID) LÀM KHÓA CHÍNH THAY VÌ SỐ NGUYÊN TĂNG TỰ ĐỘNG (AUTO-INCREMENT)?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Khóa chính UUID vs Integer	**Khó**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

⚡ Câu trả lời nhanh (20–40 giây):

UUID (128-bit) có tính duy nhất toàn cầu, không thể đoán được. Giúp chống lại kiểu tấn công duyệt ID hàng loạt (Insecure Direct Object Reference - IDOR) và cho phép client sinh ID trước khi gửi lên server.

🔍 Giải thích kỹ thuật chuyên sâu:

Nếu dùng số nguyên 1, 2, 3, kẻ tấn công chỉ cần đổi id=1 thành id=2 trên URL là duyệt hết dữ liệu của người khác. Với Guid như '7c6f3ca1-8b9a-...', không ai có thể đoán được ID tiếp theo là gì. Ngoài ra, UUID hỗ trợ phân tán dữ liệu mà không sợ trùng khóa.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Core/Entities/BaseEntity.cs: public Guid Id { get; set; } = Guid.NewGuid();

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Xem ID bài viết hoặc sự kiện trên Swagger -> Đều có định dạng UUID 36 ký tự.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho rằng UUID làm giảm hiệu năng tìm kiếm một cách nghiêm trọng ở quy mô sinh viên.

CÂU 25: CƠ CHẾ PHÂN TRANG PAGEDRESPONSE<T> HOẠT ĐỘNG NHƯ THẾ NÀO TẠI TẦNG CƠ SỞ DỮ LIỆU?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Phân trang Cơ sở dữ liệu	**Trung bình**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

⚡ Câu trả lời nhanh (20–40 giây):

Server dùng hai hàm IQueryable của LINQ: .Skip((pageIndex - 1) * pageSize) và .Take(pageSize). Entity Framework Core sẽ dịch 2 hàm này thành mệnh đề OFFSET và LIMIT trong câu lệnh SQL gửi xuống PostgreSQL.

🔍 Giải thích kỹ thuật chuyên sâu:

Nhờ vậy, server chỉ lấy đúng số dòng cần thiết (ví dụ 10 dòng) từ ổ cứng lên RAM, không bao giờ kéo toàn bộ 100.000 dòng lên rồi mới cắt. Tổng số bản ghi được đếm bằng truy vấn riêng: await query.CountAsync().

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Controllers/PostsController.cs dòng 55-60.

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Kiểm tra SQL log của EF Core -> Thấy rõ câu lệnh: SELECT ... OFFSET 10 ROWS FETCH NEXT 10 ROWS ONLY.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Gọi .ToList() trước rồi mới gọi .Skip().Take() (làm vậy là phân trang trên RAM, không phải trên DB).

CÂU 26: TẠI SAO PHẢI THÊM TIÊU CHÍ SẮP XẾP PHỤ (TIE-BREAKER) ID KHI PHÂN TRANG?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Tie-Breaker khi Sắp xếp	**Khó**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

⚡ Câu trả lời nhanh (20–40 giây):

Nếu chỉ sắp xếp theo PublishedAt hoặc StartTime, khi nhiều bản ghi có cùng một thời điểm đăng, cơ sở dữ liệu PostgreSQL không đảm bảo thứ tự trả về giữa các lần truy vấn. Việc thêm .ThenByDescending(p => p.Id) đảm bảo thứ tự tuyệt đối ổn định, không làm trùng lặp hoặc sót bản ghi khi chuyển trang.

🔍 Giải thích kỹ thuật chuyên sâu:

Đây là bài học sâu sắc từ việc audit lỗi Tin tức: khi không có tie-breaker, một bài viết ở trang 1 có thể nhảy sang trang 2 ở lần gọi tiếp theo, khiến người dùng nhìn thấy bài bị trùng lặp. Khóa chính Id luôn là duy nhất nên đóng vai trò tie-breaker hoàn hảo.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Controllers/PostsController.cs dòng 56:
.OrderByDescending(p => p.PublishedAt ?? p.CreatedAt).ThenByDescending(p => p.Id)

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Xem bài test Pagination_AcrossPages_NoDuplicatePostsAndCoversAllItems trong PostsLogicAuditTests.cs -> Pass 100%.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho rằng chỉ cần OrderBy theo ngày tháng là đủ ổn định.

CÂU 27: CHUỖI JSON WEB TOKEN (JWT) GỒM MẤY PHẦN? Ý NGHĨA CỦA TỪNG PHẦN LÀ GÌ?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Cấu trúc JWT Token	**Dễ**	Hồ Đoàng Nguyên Khang	Đào Trí Khang

Câu trả lời nhanh (20–40 giây):

JWT gồm đúng 3 phần phân tách nhau bằng dấu chấm (.): Header (Thuật toán ký & Loại token), Payload (Dữ liệu người dùng và các Claims), và Signature (Chữ ký số bí mật để bảo vệ tính toàn vẹn).

Giải thích kỹ thuật chuyên sâu:

Header: {"alg": "HS256", "typ": "JWT"}. Payload: Chứa sub (UserId), email, role, exp (hạn dùng), jti (mã định danh token). Signature: Băm Header + Payload + Secret Key bằng thuật toán HMAC-SHA256.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Infrastructure/Services/JwtService.cs.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Sao chép token khi đăng nhập, dán vào trang jwt.io -> Thấy rõ 3 phần màu đỏ (Header), màu tím (Payload) và màu xanh (Signature).

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho rằng JWT mã hóa dữ liệu làm người khác không đọc được.

CÂU 28: QUÁ TRÌNH SINH TOKEN JWT DIỄN RA Ở ĐÂU TRÊN SERVER?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Cấp phát JWT Server-Side	**Trung bình**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

Câu trả lời nhanh (20–40 giây):

Quá trình sinh token diễn ra độc quyền tại JwtService.cs trong tầng Infrastructure của server, sau khi người dùng xác thực đúng email và mật khẩu tại AuthController.

Giải thích kỹ thuật chuyên sâu:

Hàm GenerateToken() lấy thông tin User từ cơ sở dữ liệu, kiểm tra danh sách Roles của người dùng, đóng gói vào các Claim, lấy biến môi trường JWT_SECRET để ký và xuất ra chuỗi token 3 đoạn.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Infrastructure/Services/JwtService.cs dòng 25-60.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Gọi POST /api/auth/login trên Swagger -> Nhận chuỗi token dài khoảng 700 ký tự.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho rằng client tự tạo ra token.

CÂU 29: QUÁ TRÌNH KIỂM TRA VÀ GIẢI MÃ TOKEN DIỄN RA Ở ĐÂU TRÊN SERVER?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Xác thực JWT Middleware	**Trung bình**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

Câu trả lời nhanh (20–40 giây):

Diễn ra tại JwtBearerMiddleware được đăng ký trong pipeline Program.cs thông qua builder.Services.AddAuthentication().AddJwtBearer(). Middleware này tự động chặn bắt mọi request đến trước khi vào Controller.

Giải thích kỹ thuật chuyên sâu:

Middleware đọc header Authorization, tách lấy chuỗi sau chữ 'Bearer ', dùng IssuerSigningKey bí mật để xác minh chữ ký và kiểm tra hạn sử dụng (exp). Nếu hợp lệ, nó gán thông tin vào đối tượng HttpContext.User (ClaimsPrincipal). Nếu sai, nó trả 401 ngay lập tức.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Program.cs dòng 85-115.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Gọi endpoint cần quyền với token sai 1 ký tự -> Nhận ngay mã 401 Unauthorized.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Tưởng rằng lập trình viên phải tự viết hàm kiểm tra token trong từng Controller.

CÂU 30: HEADER AUTHORIZATION CÓ CẤU TRÚC NHƯ THẾ NÀO KHI GỬI TỪ CLIENT LÊN API?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Header Authorization Bearer	**Dễ**	Quách Khoa Đức	Đào Trí Khang

Câu trả lời nhanh (20–40 giây):

Header có tên là 'Authorization', và giá trị có định dạng chuẩn: 'Bearer <chuỗi_token_jwt>'. Giữa từ khóa Bearer và token bắt buộc phải có một dấu cách (space).

Giải thích kỹ thuật chuyên sâu:

Bearer là lược đồ xác thực chuẩn theo RFC 6750, có nghĩa là 'Người mang mã này có quyền truy cập'. Cả ba ứng dụng đều thêm header này vào request.

Mã nguồn minh chứng (File, Class, Method, Dòng):

Web: headers['Authorization'] = `Bearer ${token}`;
WinForms: request.Headers.Authorization = new AuthenticationHeaderValue('Bearer', token);
Android: Interceptor tự động gắn header trong ApiClient.java.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Bấm nút Authorize trên Swagger, nhập 'Bearer ...' và thực thi request -> Quan sát phần Curl command thấy rõ header này.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Quên chữ Bearer hoặc quên dấu cách khiến server không nhận diện được token.

CÂU 31: NẾU TOKEN JWT BỊ ĐÁNH CẮP THÌ KẺ TẤN CÔNG CÓ THỂ LÀM GÌ VÀ PHÒNG NGỪA RA SAO?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Bảo vệ Token bị đánh cắp	**Khó**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

⚡ Câu trả lời nhanh (20–40 giây):

Kẻ tấn công có thể mạo danh người dùng gửi request cho đến khi token hết hạn. Nhóm phòng ngừa bằng 3 lớp: (1) Đặt thời gian hết hạn ngắn (4 giờ); (2) Bắt buộc chạy HTTPS để chống nghe lén trên đường truyền; (3) Lưu trữ token an toàn và chuẩn bị cơ chế Blacklist token.

🔍 Giải thích kỹ thuật chuyên sâu:

Vì JWT là stateless nên server không lưu session. Để tăng cường bảo mật, hệ thống có thể tích hợp Refresh Token lưu trong HttpOnly Cookie và lưu mã JTI của token bị thu hồi vào Redis Cache để vô hiệu hóa tức thời khi phát hiện bất thường.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Infrastructure/Services/JwtService.cs (cấu hình DateTime.UtcNow.AddHours(4)).

🔧 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Đợi token quá 4 giờ -> Gọi API sẽ nhận 401 token expired.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho rằng JWT tồn tại vĩnh viễn không bao giờ hết hạn.

CÂU 32: MẬT KHẨU NGƯỜI DÙNG ĐƯỢC LƯU TRỮ NHƯ THẾ NÀO TRONG CƠ SỞ DỮ LIỆU?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Băm Mật khẩu BCrypt	**Trung bình**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

⚡ Câu trả lời nhanh (20–40 giây):

Mật khẩu tuyệt đối không lưu dạng văn bản rõ (plaintext) mà được băm một chiều bằng thuật toán BCrypt kèm muối ngẫu nhiên (Salt) thông qua thư viện BCrypt.Net-Next.

🔍 Giải thích kỹ thuật chuyên sâu:

BCrypt là thuật toán băm chậm có chủ đích (work factor), giúp chống lại các cuộc tấn công Brute-force và tấn công Bảng cầu vồng (Rainbow Table). Mỗi lần băm một mật khẩu giống nhau sẽ cho ra chuỗi hash khác nhau nhờ muối tự sinh ngẫu nhiên.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Infrastructure/Services/PasswordHasher.cs:
BCrypt.Net.BCrypt.HashPassword(password) và VerifyPassword(password, hash).

🔧 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở bảng Users trong PostgreSQL -> Cột PasswordHash là chuỗi 60 ký tự bắt đầu bằng $2a$11$..., không ai đọc được mật khẩu gốc.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Dùng các thuật toán băm cũ đã lỗi thời như MD5 hay SHA-1.

CÂU 33: VAI TRÒ CỦA MIDDLEWARE EXCEPTIONHANDLINGMIDDLEWARE TRONG ASP.NET CORE?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Middleware Xử lý Ngoại lệ	**Trung bình**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

Câu trả lời nhanh (20–40 giây):

Là bộ lọc bắt lỗi tập trung (Global Exception Handler) nằm ở ngoài cùng của pipeline. Bất kỳ lỗi chưa bắt nào xảy ra trong controller hay service đều được middleware này bắt lại, ghi chi tiết vào log, và trả về cho client mã lỗi 500 kèm mã sự cố duy nhất (maSuCo) dạng JSON văn minh.

Giải thích kỹ thuật chuyên sâu:

Nguyên tắc: Chi tiết đi vào log, chung chung đi ra client. Middleware ngăn chặn việc lộ vết ngăn xếp hệ thống (Stack Trace) và cấu trúc cơ sở dữ liệu ra màn hình người dùng cuối, phòng chống tin tặc khai thác thông tin.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Middleware/ExceptionHandlingMiddleware.cs.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Cố tình ném ngoại lệ throw new Exception('Lỗi test') -> Client nhận JSON { success: false, message: 'Đã xảy ra sự cố máy chủ nội bộ.', incidentId: '...' } với HTTP 500.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Để mặc định của framework bung trang vàng màn hình lỗi kèm toàn bộ code.

CÂU 34: MÃ SỰ CỐ (MASUCO) LÀ GÌ? TẠI SAO NGƯỜI DÙNG CHỈ THẤY MÃ SỰ CỐ CÒN CHI TIẾT GHI VÀO LOG?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Mã Sự cố IncidentId	**Dễ**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

Câu trả lời nhanh (20–40 giây):

Mã sự cố là một chuỗi ngẫu nhiên (ví dụ 8 ký tự hex) do server sinh ra khi có lỗi 500. Người dùng đọc mã này cho bộ phận kỹ thuật để quản trị viên tra đúng dòng lỗi trong file log của server mà không làm lộ thông tin bảo mật cho người dùng.

Giải thích kỹ thuật chuyên sâu:

Nếu hiện cả lỗi 'NpgsqlException: relation does not exist at line 55' cho sinh viên, sinh viên sẽ hoang mang và tin tặc biết được cấu trúc database. Bằng mã sự cố, người dùng chỉ thấy thông báo lịch sự, còn kỹ sư tra cứu log trong 5 giây là tìm đúng vị trí bug.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Middleware/ExceptionHandlingMiddleware.cs dòng 40-55.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Xem tài liệu học phần Trang 18 Bảng 17 và ảnh 15_real_error_log.png.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Ẩn luôn mã sự cố khiến người dùng không biết báo lỗi thế nào.

CÂU 35: TẠI SAO WEB NEXT.JS SỬ DỤNG ROUTE HANDLER /API/[...PATH] LÀM BFF PROXY?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng

BFF Proxy trên Web	**Khó**	Quách Khoa Đức	Hồ Đoàng Nguyên Khang

⚡ Câu trả lời nhanh (20–40 giây):

BFF (Backend-for-Frontend) route handler đóng vai trò cầu nối cùng nguồn gốc (Same-Origin). Trình duyệt chỉ gửi request tương đối tới /api/... của Next.js, sau đó server Node.js của Vercel mới chuyển tiếp (forward) tới Render backend. Nhờ vậy, loại bỏ hoàn toàn lỗi CORS và lỗi bảo mật Mixed Content giữa HTTP/HTTPS.

🔍 Giải thích kỹ thuật chuyên sâu:

Khi frontend chạy trên domain visionary-platform.vercel.app, việc gọi thẳng tới domain khác có thể bị các tiện ích mở rộng hoặc tường lửa trình duyệt chặn. Gọi qua relative path /api/... đảm bảo 100% cùng nguồn gốc và che giấu địa chỉ thật của backend đám mây.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: frontend/src/app/api/[...path]/route.ts.

🔧 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở F12 Network trên trang Web -> Thấy URL gọi là /api/events chứ không phải tên miền Render trực tiếp.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nghĩ rằng Next.js viết lại toàn bộ logic backend (Next.js chỉ proxy gói tin).

CÂU 36: TẠI SAO ANDROID MÁY ẢO PHẢI GỌI ĐỊA CHỈ 10.0.2.2 THAY VÌ LOCALHOST?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Địa chỉ Android 10.0.2.2	**Dễ**	Đào Trí Khang	Quách Khoa Đức

⚡ Câu trả lời nhanh (20–40 giây):

Vì máy ảo Android Emulator chạy trong một hệ thống máy ảo riêng biệt và coi địa chỉ 127.0.0.1 (localhost) là chính nó. Địa chỉ 10.0.2.2 là địa chỉ IP ảo đặc biệt do Google thiết kế để làm gateway trỏ ngược về loopback interface của máy tính host đang chạy máy ảo.

🔍 Giải thích kỹ thuật chuyên sâu:

Nếu nhập http://localhost:5000 trong app Android trên máy ảo, app sẽ tự tìm cổng 5000 bên trong hệ điều hành Android và báo lỗi không nối được cổng (Connection Refused). Nhập http://10.0.2.2:5000 thì gói tin mới được định tuyến ra ngoài máy tính Windows.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

Tài liệu căn cứ: TH71108_WebAPI_3Platform_QuyUocDatTen Trang 3 Bảng 3.

🔧 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Trên máy ảo Android, gõ http://10.0.2.2:5000 trên trình duyệt Chrome của máy ảo -> Thấy Swagger của máy tính hiện ra.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nghĩ rằng máy ảo và máy tính dùng chung một card mạng nội bộ localhost.

CÂU 37: KHI NÀO ỨNG DỤNG ANDROID NHẬN LỖI NETWORKONMAINTHREADEXCEPTION?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
NetworkOnMainThread	**Dễ**	Đào Trí Khang	Võ Ngọc Minh Đạt

⚡ Câu trả lời nhanh (20–40 giây):

Xảy ra khi lập trình viên thực hiện các thao tác truyền thông mạng (mở Socket, HTTP request, đọc InputStream) trực tiếp trên Luồng Giao diện chính (Main/UI Thread) thay vì đẩy vào luồng nền.

🔍 Giải thích kỹ thuật chuyên sâu:

Hệ điều hành Android chủ động ném ngoại lệ này để bảo vệ trải nghiệm người dùng, tránh tình trạng ứng dụng bị đóng băng khi mạng lag.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: mobile/VisionaryAndroid/.../EventsFragment.java (dùng enqueue để tránh lỗi).

🔧 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Xem chi tiết tại Slide 6 và tài liệu Q&A Nhóm B Câu 11.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cố tình dùng chính sách StrictMode để tắt lỗi này thay vì chuyển sang luồng nền.

CÂU 38: TẠI SAO ANDROID KHÔNG TIN CHỨNG CHỈ SSL TỰ KÝ CỦA MÔI TRƯỜNG LOCALHOST?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Chứng chỉ SSL trên Android	**Trung bình**	Đào Trí Khang	Hồ Đoàng Nguyên Khang

⚡ Câu trả lời nhanh (20–40 giây):

Vì chứng chỉ tự ký của .NET (dotnet dev-certs) chỉ được cấp cho tên miền 'localhost' và được cài vào chứng chỉ tin cậy của Windows, không có trong kho chứng chỉ tin cậy của Android (CA Store). Hơn nữa Android gọi qua IP 10.0.2.2 nên tên miền không khớp (Name Mismatch).

🔍 Giải thích kỹ thuật chuyên sâu:

Khắc phục: Trong môi trường phát triển cục bộ, Android bắt buộc dùng giao thức HTTP thường cổng 5000/5230. Khi triển khai production, backend dùng chứng chỉ SSL chính thức do Let's Encrypt cấp cho tên miền hợp lệ thì Android mới dùng HTTPS bình thường.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

Tài liệu căn cứ: TH71108_WebAPI_3Platform_QuyUocDatTen Trang 4 Bảng 2 và Trang 5.

🔧 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Gọi HTTPS tới IP cục bộ trên Android -> Báo lỗi SSLHandshakeException.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cố cài chứng chỉ tự ký vào hệ điều hành Android của sinh viên trong lúc thuyết trình.

CÂU 39: TẠI SAO WINFORMS KHÔNG NÊN KHỞI TẠO NEW HTTPCLIENT() TRONG MỖI HÀM? (SOCKET EXHAUSTION)

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Socket Exhaustion WinForms	**Trung bình**	Võ Ngọc Minh Đạt	Hồ Đoàng Nguyên Khang

⚡ Câu trả lời nhanh (20–40 giây):

Khởi tạo new HttpClient() liên tục sẽ làm cạn kiệt cổng mạng Socket của hệ điều hành. Khi client đóng connection, socket không giải phóng ngay mà rơi vào trạng thái TIME_WAIT trong 4 phút. Quá nhiều request liên tục sẽ làm máy tính hết cổng mạng và crash ứng dụng.

🔍 Giải thích kỹ thuật chuyên sâu:

Giải pháp chuẩn là dùng biến static readonly HttpClient duy nhất (Singleton pattern) cho toàn bộ ứng dụng để tái sử dụng connection pool.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: desktop/Visionary.Desktop/Services/ApiClient.cs dòng 16.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Xem chi tiết tại Câu 10 Nhóm B.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nghĩ rằng từ khóa using sẽ đóng socket ngay lập tức.

CÂU 40: LỖI DEADLOCK TRONG WINFORMS XẢY RA KHI NÀO VÀ CÁCH PHÒNG TRÁNH?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Deadlock trong WinForms	**Trung bình**	Võ Ngọc Minh Đạt	Quách Khoa Đức

Câu trả lời nhanh (20–40 giây):

Xảy ra khi trộn lẫn giữa lập trình đồng bộ và bất đồng bộ: gọi .Result hoặc .Wait() trên một Task đang chạy trên UI Thread. Phòng tránh bằng cách dùng xuyên suốt async và await từ sự kiện nút bấm cho tới hàm gọi mạng.

Giải thích kỹ thuật chuyên sâu:

Khi dùng await, luồng giao diện được giải phóng để xử lý các sự kiện click chuột và vẽ màn hình, sau đó SynchronizationContext tự động điều phối trả về UI thread.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: desktop/Visionary.Desktop/Forms/MainForm.cs.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Xem phân tích chi tiết tại Câu 9 Nhóm B.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Dùng Thread.Sleep() trên luồng giao diện để chờ mạng.

CÂU 41: SỰ CỐ NETSDK1100 ĐÃ XẢY RA TRONG DỰ ÁN LÀ GÌ VÀ NHÓM SỬA NHƯ THẾ NÀO?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Sự cố NETSDK1100 Đã Gặp	**Dễ**	Võ Ngọc Minh Đạt	Quách Khoa Đức

Câu trả lời nhanh (20–40 giây):

Là lỗi biên dịch khi GitHub Actions chạy lệnh 'dotnet build backend/Visionary.sln' trên máy ảo Linux Ubuntu. Do file solution ban đầu chứa cả dự án Windows Forms (.NET 8 Windows Desktop), Linux không hỗ trợ build Windows UI nên ném lỗi NETSDK1100. Nhóm đã sửa bằng cách tạo solution Visionary.Backend.sln độc lập chỉ chứa các dự án backend để CI Linux build mượt mà.

Giải thích kỹ thuật chuyên sâu:

Đây là lỗi có thật 100% được ghi lại trong log GitHub Actions. Việc tách solution là giải pháp kiến trúc Clean Architecture chuẩn mực.

Mã nguồn minh chứng (File, Class, Method, Dòng):

Minh chứng: deliverables/evidence/15_real_error_log.png và 16_error_fixed_test.png.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Xem workflow backend.yml và desktop.yml hoạt động độc lập trên GitHub.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Đổi runner của backend sang windows-latest làm chậm thời gian build CI.

CÂU 42: SỰ CỐ ANDROID LỆCH KIỂU GENERIC TYPE GIỮA APIRESPONSE<LIST<T>> VÀ PAGEDRESPONSE ĐÃ ĐƯỢC KHẮC PHỤC THẾ NÀO?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Sự cố Generic Type Android	**Trung bình**	Đào Trí Khang	Hồ Đoàng Nguyên Khang

⚡ Câu trả lời nhanh (20–40 giây):

Ban đầu interface Retrofit khai báo nhận ApiResponse<List<LearningResourceItem>>, trong khi backend C# trả về cấu trúc phân trang ApiResponse<PagedResponse<LearningResourceDto>>. Gson gặp Object khi đang chờ Array nên ném lỗi JsonParseException. Nhóm đã đồng bộ interface Android dùng đúng ApiResponse<PagedResponse<T>> và truyền tham số pageIndex, pageSize.

🔍 Giải thích kỹ thuật chuyên sâu:

Đây là minh chứng cho năng lực số 3 trong PDF: 'Ánh xạ JSON - Đọc một phản hồi thật và chuyển thành lớp của ngôn ngữ mình dùng'.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: mobile/VisionaryAndroid/.../VisionaryApiService.java dòng 40-45.

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Chạy ./gradlew assembleDebug -> Build thành công 100% không còn lỗi cú pháp.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Sửa controller backend trả về mảng thô chỉ để làm vừa lòng code Android viết ẩu.

CÂU 43: TẠI SAO HỆ THỐNG LOẠI BỎ HOÀN TOÀN DỮ LIỆU FALLBACK GIẢ TRÊN PRODUCTION?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Loại bỏ Fallback Data	**Dễ**	Quách Khoa Đức	Hồ Đoàng Nguyên Khang

⚡ Câu trả lời nhanh (20–40 giây):

Việc dùng dữ liệu mẫu (mock/fallback) khi API rỗng hoặc lỗi là một sai lầm nghiêm trọng che giấu lỗi thật và vi phạm tính toàn vẹn dữ liệu. Nhóm đã loại bỏ 100% fallback để hệ thống minh bạch: rỗng thì hiển thị Empty State văn minh, lỗi thì hiển thị Error State kèm nút Thử lại.

🔍 Giải thích kỹ thuật chuyên sâu:

Nếu giữ fallback, khi có sự cố mạng, người dùng vẫn thấy dữ liệu giả và tưởng hệ thống đang chạy bình thường, dẫn đến thao tác sai lệch.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

Minh chứng: frontend/src/app/posts/page.tsx và achievements/page.tsx.

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Tìm kiếm từ khóa không có thật -> Thấy thông điệp 'Chưa có nội dung phù hợp với bộ lọc', không thấy bài viết giả.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Nghĩ rằng trang web có dữ liệu giả trông đẹp hơn là trang web rỗng.

CÂU 44: ĐẲNG THỨC BẤT BIẾN PHÂN LOẠI TIN TỨC (CATEGORY UNION INVARIANT) ĐƯỢC KIỂM CHỨNG RA SAO?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Đẳng thức Bất biến Tin tức	**Trung bình**	Quách Khoa Đức	Hồ Đoàng Nguyên Khang

⚡ Câu trả lời nhanh (20–40 giây):

Bất biến: Tổng số bài viết của tất cả các danh mục con cộng lại phải bằng đúng số bài viết hiển thị trong tab 'Tất cả danh mục': PublishedPosts(All) = UNION(PublishedPosts(Category)). Nhóm kiểm chứng trên cơ sở dữ liệu thật: 13 bài Công nghệ + 6 bài Tiếng Anh + 5 bài Tin tức + 4 bài CLB = đúng 28 bài trong tab Tất cả.

🔍 Giải thích kỹ thuật chuyên sâu:

Bất biến này đã được nhóm viết thành bài test xUnit tự động CategoryUnionInvariant_SumOfCategories_EqualsAllPublishedPosts chạy tự động mỗi khi commit.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File test: backend/tests/Visionary.Tests/PostsLogicAuditTests.cs.

🔧 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở web chọn tab Tất cả (28 bài), chọn từng tab con cộng lại đúng 28 bài duy nhất.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Đổi danh mục làm xuất hiện các bài viết không có trong tập Tất cả.

CÂU 45: LÀM SAO ĐẢM BẢO CÁC BÀI VIẾT NHÁP (DRAFT) KHÔNG BAO GIỜ BỊ RÒ RỈ RA API CÔNG KHAI?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Bảo vệ Bài viết Nháp	**Trung bình**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

⚡ Câu trả lời nhanh (20–40 giây):

Server áp dụng bộ lọc bảo mật ở hai cấp: Trong danh sách GET /api/posts, server luôn gắn điều kiện p.Status == PostStatus.Published. Trong chi tiết theo slug GET /api/posts/{slug}, nếu bài là Draft hoặc PendingReview, server kiểm tra quyền; người dùng vãng lai hoặc sinh viên sẽ nhận HTTP 404 Not Found ngay lập tức.

🔍 Giải thích kỹ thuật chuyên sâu:

Nhóm không bao giờ tải bài nháp về client rồi dùng CSS ẩn đi. Bảo mật bắt buộc phải thực thi tại truy vấn SQL phía server.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Controllers/PostsController.cs dòng 52 và dòng 120-135.

🔧 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Chạy test GetPostBySlug_DraftPost_ReturnsNotFoundForGuest -> Pass 100%.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Lấy slug bài nháp chia sẻ cho người khác vẫn đọc được.

CÂU 46: TẠI SAO CƠ SỞ DỮ LIỆU POSTGRESQL ĐẶT TẠI SINGAPORE LẠI CẦN CONNECTION POOLER IPV4?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
PostgreSQL Connection	**Khó**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

Pooler

Câu trả lời nhanh (20–40 giây):

Vì tên miền trực tiếp của Supabase là IPv6-only trên gói miễn phí, trong khi môi trường máy chủ đám mây của Render chỉ hỗ trợ mạng outbound IPv4. Kết nối trực tiếp sẽ bị lỗi timeout. Nhóm đã chuyển sang Supabase Connection Pooler (cổng 5432) hỗ trợ IPv4 đầy đủ.

Giải thích kỹ thuật chuyên sâu:

Connection Pooler cũng giúp tối ưu số lượng kết nối đồng thời tới PostgreSQL, tránh quá tải tiến trình cơ sở dữ liệu khi có nhiều request đồng thời.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: render.yaml và cấu hình chuỗi kết nối Npgsql trong appsettings.json.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Backend trên Render kết nối Supabase thành công 100%, endpoint /health/ready trả về Ready.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Đổ lỗi cho Supabase bị sập khi không nối được cổng.

CÂU 47: LÀM SAO ĐỂ PHÂN QUYỀN THEO CHỦ SỞ HỮU (OWNERSHIP AUTHORIZATION) KHI SINH VIÊN ĐĂNG KÝ SỰ KIỆN?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Phân quyền theo Chủ sở hữu	**Trung bình**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

Câu trả lời nhanh (20–40 giây):

Server đọc UserId từ token JWT đã xác thực, sau đó kiểm tra trong bảng EventRegistrations xem UserId này đã đăng ký sự kiện này chưa. Nếu sinh viên muốn hủy đăng ký, server chỉ cho phép hủy nếu bản ghi đăng ký đó thuộc về chính UserId của người gọi.

Giải thích kỹ thuật chuyên sâu:

Thuộc tính [Authorize] chỉ kiểm tra có đăng nhập hay chưa. Logic kiểm tra 'bản ghi này có phải của tôi không' phải viết bằng code C# trong controller.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Controllers/EventsController.cs dòng 250-268.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Đăng nhập tài khoản A nhưng cố tình hủy đăng ký của tài khoản B -> Server từ chối.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho phép client truyền RegistrationId của người khác lên để xóa.

CÂU 48: NẾU HỆ THỐNG PHẢI MỞ RỘNG PHỤC VỤ 10.000 SINH VIÊN TRUY CẬP CÙNG LÚC THÌ NHÓM SẼ TỐI ƯU NHỮNG GÌ?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Khả năng Mở rộng 10.000 User	**Khó**	Quách Khoa Đức	Hồ Đoàng Nguyên Khang

⚡ Câu trả lời nhanh (20–40 giây):

Nhóm sẽ áp dụng 4 giải pháp: (1) Caching đa tầng: Tích hợp Redis Cache cho các dữ liệu ít thay đổi như danh sách sự kiện và tin tức; (2) CDN & Edge Caching trên Vercel/Cloudflare; (3) Tối ưu hóa chỉ mục Index trên PostgreSQL cho các cột tìm kiếm và khóa ngoại; (4) Mở rộng theo chiều ngang (Horizontal Scaling) cho máy chủ Web API nhờ kiến trúc Stateless của JWT.

🔍 Giải thích kỹ thuật chuyên sâu:

Vì Web API không lưu session trong bộ nhớ, chúng ta có thể dễ dàng đặt 5 hoặc 10 container API phía sau một Load Balancer (Nginx hoặc AWS ALB) mà không sợ mất trạng thái đăng nhập của sinh viên.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Infrastructure/Services/CacheService.cs.

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Kiến trúc Stateless của JWT và phân tách tầng DTO hiện tại đã sẵn sàng 100% cho việc mở rộng.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Trả lời chung chung kiểu 'mua máy chủ mạnh hơn' (Scale up tốn kém và có giới hạn).

CHƯƠNG ĐẶC BIỆT: SỰ CỐ BACKEND CI #12 – TỪ 13/15 ĐẾN KẾT QUẢ KIỂM THỬ MỚI

Chương này tập trung toàn bộ các câu hỏi chuyên sâu về sự cố kỹ thuật có thật trên GitHub Actions Backend CI Run #12 (Commit 2153dd1), quy trình phân tích nguyên nhân gốc rễ (Root Cause Analysis), giải pháp kiến trúc hai tầng và kết quả kiểm chứng hồi quy 15/15 Passed trên GitHub Actions Run #40231783994 (Commit c7af080). Đây là bằng chứng sống động nhất về năng lực tự sửa lỗi thực tế của nhóm theo rubric TH71108.

CÂU 49: VÌ SAO TRÊN GITHUB ACTIONS BÁO 'BUILD FAILED' NHƯNG LẠI GHI '0 WARNING(S) 0 ERROR(S)'?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
CI Build Failed vs Compiler Errors	**Cơ bản**	Võ Ngọc Minh Đạt	Hồ Đoàng Nguyên Khang

⚡ Câu trả lời nhanh (20–40 giây):

Vì mã nguồn C# đã được biên dịch hoàn toàn thành công (0 lỗi cú pháp), nhưng lệnh 'dotnet test' sau đó thất bại do có 2 bài kiểm thử xUnit không vượt qua. Trong quy trình CI/CD chuyên nghiệp, dotnet test trả về Exit Code 1 khi có bất kỳ bài test nào fail, khiến runner đánh dấu cả step và workflow là thất bại.

🔍 Giải thích kỹ thuật chuyên sâu:

Nhiều sinh viên nhầm lẫn giữa lỗi biên dịch (Compile Error) và lỗi kiểm thử hồi quy (Test Assertion Failure). Trình biên dịch Roslyn hoàn thành 100% với 0 lỗi, nhưng xUnit Runner ném lỗi khi Assert.True() và HttpRequestException gặp sự cố logic.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: .github/workflows/backend-ci.yml bước 'Run unit & integration tests'.

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Log Run #12: 'Total tests: 15. Passed: 13. Failed: 2. Process completed with exit code 1.'

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho rằng code bị lỗi cú pháp hoặc thiếu thư viện nuget.

CÂU 50: CHỈ RÕ CHÍNH XÁC TÊN, FILE VÀ DÒNG BÁO LỖI CỦA HAI BÀI TEST BỊ THẤT BẠI TRONG CI RUN #12?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Hai bài Test thất bại trong Run #12	**Cơ bản**	Võ Ngọc Minh Đạt	Hồ Đoàng Nguyên Khang

Câu trả lời nhanh (20–40 giây):

Bài 1: Visionary.Tests.PostsLogicAuditTests.Search_WithCategory_And_EmptyState_AreAccurate tại dòng 168, thông báo 'Assert.True() Failure - Expected: True, Actual: False' (TotalCount = 0). Bài 2: Visionary.Tests.PostsLogicAuditTests.GetPostBySlug_ValidPublishedPost_Returns200AndIncrementsView tại dòng 183, thông báo 'System.Net.Http.HttpRequestException: 404 (Not Found)'.

Giải thích kỹ thuật chuyên sâu:

Cả hai bài test đều thuộc cùng test class PostsLogicAuditTests.cs và đều kiểm thử các endpoint đọc dữ liệu bài viết đã được seed vào database.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/tests/Visionary.Tests/PostsLogicAuditTests.cs dòng 168 và dòng 183.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mở log nguyên văn GitHub Actions Run #12 bước xUnit test.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Đọc nhầm sang các test của ApiIntegrationTests.

CÂU 51: VÌ SAO TEST TÌM KIẾM TỪ KHÓA 'CLEAN ARCHITECTURE' LẠI TRẢ VỀ TOTALCOUNT = 0 DÙ TRONG CODE SEED CÓ BÀI VIẾT NÀY?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nguyên nhân Search không ra kết quả	**Khó**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

Câu trả lời nhanh (20–40 giây):

Không phải do thuật toán tìm kiếm hay ToLower() bị sai, mà do chu trình SeedAsync() của DbInitializer đã bị hủy ngang (aborted) giữa chừng ở bước khởi tạo Role do đụng độ khóa ngoại/ràng buộc duy nhất với tiến trình test chạy song song. Hậu quả là bảng PostCategories và Posts chưa kịp được nạp vào cơ sở dữ liệu test, dẫn tới database rỗng và tìm kiếm trả về 0 kết quả.

Giải thích kỹ thuật chuyên sâu:

Nhóm đã kiểm tra thực tế: Logic lọc trong PostsController p.Title.ToLower().Contains(search) hoàn toàn đúng; nguyên nhân là do dữ liệu seed chưa hề vào được database SQLite lúc test chạy.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Infrastructure/Data/DbInitializer.cs dòng 48-75.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Khi SeedAsync hoàn tất đủ, truy vấn /api/posts?search=Clean%20Architecture trả về đúng bài viết kiến trúc Clean Architecture.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Vội vàng sửa controller hoặc đổi chuỗi tìm kiếm trong khi nguyên nhân nằm ở chu trình seed database.

CÂU 52: VÌ SAO ENDPOINT GET /API/POSTS/CHAO-MUNG-HE-SINH-THAI-VISIONARY-2026 TRẢ VỀ HTTP 404?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Nguyên nhân Slug trả 404 Not Found	**Khó**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

Câu trả lời nhanh (20–40 giây):

Tương tự bài search, bài viết với slug 'chao-mung-he-sinh-thai-visionary-2026' chưa được nạp vào bảng Posts do chu trình seed bị ngắt sớm. Khi controller gọi _context.Posts.FirstOrDefaultAsync(p => p.Slug == slug), kết quả trả về null và controller trả đúng chuẩn RESTful là NotFound(404).

Giải thích kỹ thuật chuyên sâu:

Ngoại lệ HttpRequestException: 404 xảy ra tại client test vì GetFromJsonAsync<T>() mặc định ném ngoại lệ khi mã trạng thái HTTP không thuộc dải 2xx Success.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Controllers/PostsController.cs dòng 110-125.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Khi database được seed độc lập, endpoint trả về HTTP 200 OK cùng dữ liệu bài viết đầy đủ.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Bắt try-catch trong controller để trả 200 giả mạo.

CÂU 53: THÔNG ĐIỆP 'SQLITE ERROR 19: UNIQUE CONSTRAINT FAILED: ROLES.NAME' NGHĨA LÀ GÌ VÀ TẠI SAO LẠI XUẤT HIỆN?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Lỗi UNIQUE constraint Roles.Name	**Khó**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

Câu trả lời nhanh (20–40 giây):

Trong xUnit, hai class test ApiIntegrationTests và PostsLogicAuditTests được chạy song song theo cơ chế mặc định. Cả hai class cùng khởi tạo WebApplicationFactory trỏ vào cùng file 'visionary.db'. Khi cả hai đồng thời gọi Program.cs -> DbInitializer.SeedAsync(), cả hai luồng cùng đọc thấy bảng Roles chưa có role 'Admin', rồi cả hai cùng phát lệnh INSERT. Luồng thứ hai đến sau bị SQLite chặn lại bởi ràng buộc duy nhất (Unique Index) trên cột Name của bảng Roles.

Giải thích kỹ thuật chuyên sâu:

Exception này không được cô lập, khiến task seeding của factory thứ hai bị dừng ngay lập tức, bỏ qua toàn bộ phần seed Categories và Posts phía sau.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Infrastructure/Data/DbInitializer.cs dòng 52.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Log Run #12: 'Microsoft.Data.Sqlite.SqliteException (0x80004005): SQLite Error 19: UNIQUE constraint failed: Roles.Name at DbInitializer.SeedAsync'.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho rằng database production bị lỗi trùng tên role.

CÂU 54: VÌ SAO XUẤT HIỆN LỖI 'SQLITE ERROR 1: TABLE ACHIEVEMENTCATEGORIES ALREADY EXISTS'?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Lỗi Table already exists	**Trung bình**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

Câu trả lời nhanh (20–40 giây):

Do trong mã nguồn khởi tạo cũ gọi trực tiếp databaseCreator.CreateTablesAsync() mà không kiểm tra xem bảng đã tồn tại hay chưa. Khi factory thứ hai khởi động, schema đã được factory thứ nhất tạo xong, lệnh CREATE TABLE của SQLite ném lỗi Table already exists.

Giải thích kỹ thuật chuyên sâu:

Nhóm đã khắc phục bằng cách kiểm tra điều kiện bảo vệ: if (!await databaseCreator.HasTablesAsync()) mới gọi CreateTablesAsync().

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Infrastructure/Data/DbInitializer.cs dòng 25-32.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Sau khi thêm kiểm tra HasTablesAsync, log hoàn toàn sạch bóng lỗi SQLite Error 1.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Xóa bảng mỗi khi khởi động ứng dụng (làm mất toàn bộ dữ liệu thật).

CÂU 55: WEBAPPLICATIONFACTORY<PROGRAM> TRONG ASP.NET CORE INTEGRATION TESTING DÙNG ĐỂ LÀM GÌ?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Vai trò của WebApplicationFactory	**Trung bình**	Quách Khoa Đức	Hồ Đoàng Nguyên Khang

Câu trả lời nhanh (20–40 giây):

WebApplicationFactory<Program> là lớp kiểm thử tích hợp chính thức của Microsoft trong gói Microsoft.AspNetCore.Mvc.Testing. Nó cho phép dựng toàn bộ máy chủ Web API trên TestServer trong bộ nhớ (In-Memory), nạp đầy đủ Dependency Injection, Middleware Pipeline, Routing và Authentication mà không cần phải chạy process server riêng biệt hay mở cổng socket thật.

Giải thích kỹ thuật chuyên sâu:

Nó cho phép nhóm tạo ra đối tượng HttpClient (_client) để gửi các yêu cầu HTTP thật tới các Controller và kiểm tra phản hồi từ đầu tới cuối.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/tests/Visionary.Tests/CustomWebApplicationFactory.cs.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Kiểm thử tích hợp chạy nhanh (dưới 3 giây cho 15 test) trên mọi môi trường Windows và Linux.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Dùng Mock giả lập tất cả controller thay vì kiểm thử tích hợp thật.

CÂU 56: GIẢI PHÁP CÔ LẬP CƠ SỞ DỮ LIỆU KIỂM THỬ (TEST DATABASE ISOLATION) CỦA NHÓM HOẠT ĐỘNG RA SAO?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Khái niệm Test Database Isolation	**Khó**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

⚡ Câu trả lời nhanh (20–40 giây):

Nhóm triển khai giải pháp hai tầng kiến trúc: Tầng 1: Trong DbInitializer.cs, nhóm đặt một khóa đồng bộ luồng tĩnh private static readonly SemaphoreSlim _seedLock = new(1, 1) bọc quanh toàn bộ phương thức SeedAsync() để đảm bảo chỉ một tiến trình được phép khởi tạo database tại một thời điểm. Tầng 2: Tạo lớp CustomWebApplicationFactory kế thừa WebApplicationFactory, ghi đè ConfigureWebHost để cấu hình mỗi lần test sử dụng một file SQLite tạm biệt lập visionary_test_{Guid:N}.db và dọn dẹp sau khi chạy xong.

🔍 Giải thích kỹ thuật chuyên sâu:

Đồng thời, nhóm gom cả hai test class vào cùng một [Collection('IntegrationTests')] để xUnit chạy tuần tự trong cùng collection, triệt tiêu 100% race condition.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/tests/Visionary.Tests/CustomWebApplicationFactory.cs và DbInitializer.cs dòng 16-35.

🔧 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Chạy lặp lại bộ test 10 lần liên tiếp đều đạt 15/15 Passed tuyệt đối.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Dùng chung file database visionary.db của production cho môi trường test.

CÂU 57: SEED IDEMPOTENT LÀ GÌ VÀ VÌ SAO BẮT BUỘC TRONG HỆ THỐNG WEB API?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Khái niệm Idempotent Seeding	**Trung bình**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

⚡ Câu trả lời nhanh (20–40 giây):

Idempotent (Tính khả lũy) nghĩa là chạy thao tác đó một lần hay một ngàn lần thì kết quả trạng thái hệ thống vẫn không thay đổi và không sinh ra lỗi. Trong DbInitializer, hàm seed phải kiểm tra xem bản ghi đã tồn tại chưa (qua AnyAsync hoặc FindAsync), nếu chưa có mới INSERT, nếu đã có rồi thì bỏ qua hoặc cập nhật an toàn.

🔍 Giải thích kỹ thuật chuyên sâu:

Nếu seed không idempotent, mỗi lần khởi động lại server hoặc restart container, ứng dụng sẽ bị crash do trùng khóa chính hoặc dữ liệu bị nhân đôi (duplication).

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Infrastructure/Data/DbInitializer.cs các nhánh kiểm tra AnyAsync().

🔧 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Khởi động lại Web API trên Render nhiều lần, số lượng bài viết và sự kiện vẫn giữ nguyên chính xác.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Viết lệnh context.AddRange(...) trực tiếp mà không kiểm tra dữ liệu đã có hay chưa.

CÂU 58: TẠI SAO TUYỆT ĐỐI KHÔNG ĐƯỢC DÙNG TRY-CATCH RỖNG (SWALLOW EXCEPTION) TRONG QUÁ TRÌNH KHỞI TẠO DỮ LIỆU?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng

Tại sao không được nuốt lỗi Seed	**Khó**	Võ Ngọc Minh Đạt	Hồ Đoàng Nguyên Khang

⚡ Câu trả lời nhanh (20–40 giây):

Nếu nuốt lỗi bằng cách bắt exception rồi chỉ ghi log hoặc bỏ qua, hệ thống sẽ tiếp tục khởi động ở trạng thái dữ liệu dở dang (Inconsistent State). Server báo Ready nhưng bên trong không có Roles, không có Categories và không có Posts. Người dùng và các bài test sẽ gặp các lỗi 404, 500 bí ẩn rất khó debug.

🔍 Giải thích kỹ thuật chuyên sâu:

Nhóm tuân thủ nguyên tắc 'Fail Fast': Nếu khởi tạo schema hoặc dữ liệu nền tảng thất bại, phải ném lỗi để người quản trị biết ngay lập tức, không để API chạy trong trạng thái hỏng.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: DbInitializer.cs có transaction bảo vệ và log rõ ràng.

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Sau khi sửa, toàn bộ lỗi được xử lý có kiểm soát và log ghi nhận đầy đủ.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Bọc toàn bộ hàm main trong try-catch rỗng để che giấu lỗi.

CÂU 59: VÌ SAO KHÔNG NÊN CẤU HÌNH INTEGRATION TESTS TRỎ THẲNG VÀO POSTGRESQL PRODUCTION TRÊN SUPABASE?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Tại sao không dùng DB Production để test	**Trung bình**	Võ Ngọc Minh Đạt	Quách Khoa Đức

⚡ Câu trả lời nhanh (20–40 giây):

Vì 3 lý do: (1) An toàn dữ liệu: Integration test có các thao tác tạo mới, sửa, xóa dữ liệu; nếu trỏ vào production sẽ làm rác hoặc mất dữ liệu thật của người dùng; (2) Tốc độ và độ tin cậy: Test cục bộ qua SQLite chạy trong 1-2 giây, trong khi kết nối mạng tới cloud database ở Singapore sẽ chậm và có thể fail do mạng chập chờn; (3) Tính độc lập: CI runner trên GitHub Actions cần tự chạy độc lập mà không phụ thuộc vào kết nối mạng ngoài hay tài khoản cloud.

🔍 Giải thích kỹ thuật chuyên sâu:

Kiểm thử tích hợp đúng chuẩn luôn sử dụng In-Memory DB, Test SQLite biệt lập hoặc Testcontainers.

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/tests/Visionary.Tests/CustomWebApplicationFactory.cs.

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: GitHub Actions chạy trên Ubuntu runner không cần mở port mạng ra ngoài vẫn test trọn vẹn 15 bài.

⚠ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Hardcode connection string của Supabase vào file kiểm thử.

CÂU 60: LÀM SAO PHÂN BIỆT MỘT LỖI HTTP 404 DO LOGIC ĐÚNG VỚI HTTP 404 DO THIẾU DỮ LIỆU SEED?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Khác nhau giữa 404 thật và	**Trung bình**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

404 do thiếu seed

Câu trả lời nhanh (20–40 giây):

HTTP 404 đúng logic nghiệp vụ xảy ra khi client cố tình yêu cầu một slug không hề tồn tại (ví dụ: /api/posts/slug-khong-ton-tai-12345) hoặc cố tình đọc một bài viết đang ở trạng thái Nháp (Draft) mà không có quyền Admin. Còn HTTP 404 do thiếu dữ liệu seed xảy ra khi client gọi một slug chuẩn mực vốn phải tồn tại nhưng database test lại hoàn toàn rỗng.

Giải thích kỹ thuật chuyên sâu:

Nhóm phân biệt bằng bài test riêng: GetPostBySlug_NonExistentPost_Returns404 (mong đợi 404) và GetPostBySlug_ValidPublishedPost_Returns200AndIncrementsView (mong đợi 200).

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/tests/Visionary.Tests/PostsLogicAuditTests.cs dòng 175-195.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Sau khi sửa seed, bài slug thật trả về 200, còn slug giả trả về 404 chính xác.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Đổi Assert kiểm tra 200 thành 404 để test pass giả tạo.

CÂU 61: HIỆN TƯỢNG 'MỘT BÀI TEST CHẠY RIÊNG LẺ THÌ PASS NHƯNG CHẠY CHUNG CẢ BỘ TEST THÌ FAIL' PHẢN ÁNH ĐIỀU GÌ?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Test PASS riêng lẻ nhưng FAIL cả suite	**Khó**	Võ Ngọc Minh Đạt	Hồ Đoàng Nguyên Khang

Câu trả lời nhanh (20–40 giây):

Đây là triệu chứng kinh điển của 'Test Pollution' (Nhiễm bẩn dữ liệu test) hoặc 'Race Condition' (Xung đột tài nguyên dùng chung). Khi chạy riêng một test, nó độc chiếm file SQLite nên không ai tranh chấp. Nhưng khi chạy cả suite, xUnit kích hoạt chế độ chạy song song đa luồng giữa các test classes, dẫn tới hai WebApplicationFactory cùng tranh chấp ghi dữ liệu vào một file db duy nhất.

Giải thích kỹ thuật chuyên sâu:

Giải pháp là phải cô lập môi trường của từng test fixture hoặc gộp chung Collection để đồng bộ hóa tài nguyên.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: [Collection('IntegrationTests')] trên hai test class.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Chạy lệnh có filter chỉ một test thì pass, bỏ filter chạy cả sln thì fail; sau khi thêm lock và collection thì cả hai cách đều pass 100%.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Tắt bỏ xUnit hoặc xóa bài test bị fail.

CÂU 62: VÌ SAO NHÓM KHÔNG CHỌN CÁCH SỬA 'ASSERT.TRUE(SEARCHRES.DATA.TOTALCOUNT >= 1)' THÀNH '>= 0' HOẶC BỎ QUA BÀI TEST ĐỂ CI XANH NHANH?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng

Tuyệt đối không sửa Assert để làm xanh CI	**Trung bình**	Quách Khoa Đức	Võ Ngọc Minh Đạt

Câu trả lời nhanh (20–40 giây):

Vì việc sửa Assert thành '>= 0' hoặc thêm 'continue-on-error: true' vào GitHub Actions là hành vi 'lừa dối kiểm thử' (False Green). Nó làm pipeline hiện màu xanh nhưng thực chất tính năng tìm kiếm của hệ thống đã bị gãy hoàn toàn. Người dùng thật tìm kiếm sẽ không ra bài viết nào.

Giải thích kỹ thuật chuyên sâu:

Mục tiêu của kiểm thử hồi quy là bảo vệ chất lượng phần mềm. Thất bại trên CI là một tín hiệu quý giá chỉ ra lỗi kiến trúc cần khắc phục tận gốc rễ chứ không phải che giấu.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: Giữ nguyên vẹn toàn bộ 15 bài test và các câu lệnh Assert gốc.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Cả 15 test đều pass với điều kiện kiểm tra nghiêm ngặt nguyên bản.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Thêm || true vào câu lệnh dotnet test trên CI.

CÂU 63: NHỮNG BẰNG CHỨNG KỸ THUẬT NÀO CHỨNG MINH NHÓM ĐÃ KHẮC PHỤC TRIỆT ĐỂ VÀ MINH BẠCH SỰ CỐ CI?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Bằng chứng kỹ thuật CI đã được sửa thật	**Cơ bản**	Võ Ngọc Minh Đạt	Quách Khoa Đức

Câu trả lời nhanh (20–40 giây):

Nhóm có 4 bằng chứng kỹ thuật không thể chối cãi: (1) GitHub Actions Run ID #40231783994 trên nhánh main có trạng thái 'Success' màu xanh hoàn toàn; (2) Commit SHA c7af080 thể hiện rõ các thay đổi trong DbInitializer.cs và CustomWebApplicationFactory.cs; (3) Log kiểm thử ghi nhận: 'Total tests: 15. Passed: 15. Failed: 0. Duration: 2.8s'; (4) Ảnh chụp màn hình log lỗi cũ Run #12 và log thành công mới Run #40231783994 được đưa trực tiếp vào Slide 15 của bài thuyết trình.

Giải thích kỹ thuật chuyên sâu:

Tất cả đều công khai trên repository GitHub của nhóm tại lovemvaf/visionary-api.

Mã nguồn minh chứng (File, Class, Method, Dòng):

URL: https://github.com/lovemvaf/visionary-api/actions/runs/40231783994.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Giảng viên có thể mở link GitHub Actions kiểm tra trực tiếp ngày giờ và commit log.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Chỉ chụp ảnh màn hình terminal trên máy cá nhân mà không có link CI từ xa.

CÂU 64: CÔNG CỤ AI ĐÃ HỖ TRỢ NHÓM NHỮNG GÌ TRONG QUÁ TRÌNH PHÂN TÍCH VÀ SỬA LỖI CI NÀY?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Vai trò của AI trong quá trình	**Trung bình**	Quách Khoa Đức	Võ Ngọc Minh Đạt

sửa lỗi

Câu trả lời nhanh (20–40 giây):

Nhóm khai báo minh bạch: AI hỗ trợ phân tích log stack trace từ GitHub Actions, gợi ý nguyên nhân tiềm ẩn về xung đột Semaphore trên SQLite và đề xuất cấu trúc mẫu CustomWebApplicationFactory. Tuy nhiên, toàn bộ việc tái hiện lỗi cục bộ, quyết định thiết kế kiến trúc khóa luồng SemaphoreSlim(1,1), tích hợp vào DbInitializer.cs, chạy kiểm thử hồi quy và đẩy commit lên GitHub đều do các thành viên nhóm trực tiếp thực hiện và chịu trách nhiệm 100%.

Giải thích kỹ thuật chuyên sâu:

Nhóm không sao chép mù quáng code của AI mà luôn kiểm chứng qua câu lệnh dotnet test trước khi commit.

Mã nguồn minh chứng (File, Class, Method, Dòng):

Tài liệu: Báo cáo sự cố docs/TH71108_REAL_CI_INCIDENT_REPORT.md và khai báo AI tại Slide 1.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Mọi dòng code sửa đều được giải thích cặn kẽ về mặt kỹ thuật.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Phủ nhận hoàn toàn việc dùng AI hoặc đổ lỗi mọi thứ do AI làm.

CÂU 65: TẠI SAO BÀI VIẾT Ở TRẠNG THÁI NHÁP (DRAFT) KHÔNG ĐƯỢC PHÉP TRẢ VỀ CHO NGƯỜI DÙNG BÌNH THƯỜNG?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Bảo vệ bài viết Draft trong hệ thống	**Trung bình**	Hồ Đoàng Nguyên Khang	Quách Khoa Đức

Câu trả lời nhanh (20–40 giây):

Bài viết nháp là nội dung chưa hoàn thiện, có thể chứa thông tin nội bộ hoặc chưa được ban chủ nhiệm phê duyệt. Nếu để lộ bài nháp ra ngoài, uy tín của câu lạc bộ có thể bị ảnh hưởng. Về mặt bảo mật thông tin, đây là lỗi rò rỉ dữ liệu (Information Disclosure).

Giải thích kỹ thuật chuyên sâu:

Do đó, trong PostsController, mọi truy vấn từ khách hoặc sinh viên đều bị ép cứng điều kiện status == PostStatus.Published ngay tại câu lệnh SQL.

Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Api/Controllers/PostsController.cs dòng 52 và dòng 120.

Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Test UnpublishPost_Draft_And_PendingReview_NeverLeakPublicly -> Pass 100%.

Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Cho rằng bài nháp chỉ cần ẩn nút trên giao diện là đủ an toàn.

CÂU 66: VÌ SAO THUẬT TOÁN PHÂN TRANG DỮ LIỆU TRONG WEB API BẮT BUỘC PHẢI CÓ THỨ TỰ SẮP XẾP ỔN ĐỊNH (STABLE ORDERING)?

Phân loại	Mức độ	Người trả lời chính	Người dự phòng
Tính ổn định của thuật toán phân trang	**Khó**	Hồ Đoàng Nguyên Khang	Võ Ngọc Minh Đạt

⚡ Câu trả lời nhanh (20–40 giây):

Nếu không có mệnh đề OrderBy ổn định (ví dụ sắp xếp theo Id hoặc CreatedAt), cơ sở dữ liệu quan hệ không đảm bảo thứ tự trả về giữa các lần truy vấn. Khi người dùng bấm sang trang 2, cơ sở dữ liệu có thể trả lại một số bản ghi đã xuất hiện ở trang 1 hoặc bỏ sót các bản ghi chưa duyệt. Đây gọi là lỗi 'Pagination Inconsistency'.

🔍 Giải thích kỹ thuật chuyên sâu:

Trong EventService và PostService, nhóm luôn sắp xếp cố định theo CreatedAt giảm dần kết hợp Id trước khi gọi .Skip().Take().

💻 Mã nguồn minh chứng (File, Class, Method, Dòng):

File: backend/src/Visionary.Infrastructure/Services/EventService.cs và PostService.cs.

🧪 Cách kiểm chứng thực tế (Swagger / UI / Database):

Kiểm chứng: Duyệt từ trang 1 đến trang 3, các bản ghi hiển thị nối tiếp nhau chuẩn xác 100%, không bị lặp hay nhảy bản ghi.

⚠️ Sai lầm thường gặp & Lưu ý phản biện:

Sai lầm: Gọi .Skip().Take() trực tiếp trên DbSet mà không có .OrderBy().